

Práctica 5 - Capa de Transporte (Parte 1)

1. ¿Cuál es la función de la capa de transporte?

Su principal función es ser la comunicación lógica entre dos procesos que se encuentran en diferentes hosts. La capa de Red se encarga de la comunicación entre hosts y la de Transporte entre procesos.

2. Describa la estructura del segmento TCP y UDP.

La unidad de datos de la capa de transporte se denomina generalmente segmento (aplicable tanto a TCP como a UDP, aunque a los paquetes UDP a veces se les llama datagramas).

La principal diferencia entre uno y otro: el UDP es Transporte SIN Conexión (Ofrece el servicio mínimo: multiplexación/demultiplexación y un mecanismo de comprobación de errores (checksum). No garantiza la entrega, el orden, ni la integridad de los datos) y TCP es Orientado a CONEXIÓN (Proporciona transferencia de datos fiable, control de flujo, y control de congestión, además de multiplexación/demultiplexación. Requiere un procedimiento de establecimiento de conexión ("acuerdo en tres fases") antes de la transferencia de datos)

3. ¿Cuál es el objetivo del uso de puertos en el modelo TCP/IP?

El objetivo fundamental del uso de puertos en la Capa de Transporte es la **multiplexación y demultiplexación**.

Este mecanismo es esencial para extender el servicio de entrega que proporciona la capa de red (que es de host a host) a un servicio de comunicación lógica entre procesos de aplicación que se ejecutan en hosts diferentes.

En resumen, los puertos son la herramienta que permite a la Capa de Transporte asignar datos de la red a procesos de aplicación específicos que se ejecutan en los sistemas terminales.

4. Compare TCP y UDP en cuanto a:

TCP: es el Protocolo de Control de Transmisión.

UDP: es el Protocolo de Datagramas de Usuario

a. Confiabilidad

TCP: es fiable. Proporciona un servicio de transferencia de datos fiable.

UDP: no es fiable. Proporciona un servicio mínimo. Los segmentos UDP pueden perderse o ser entregados fuera de orden a la aplicación

b. Multiplexación.

La diferencia en cuanto a la multiplexación y demultiplexación radica fundamentalmente en la manera en que cada protocolo identifica y dirige los segmentos a los sockets de destino

UDP es un protocolo sin conexión. Su demultiplexación se considera sin conexión.

Un socket UDP se identifica completamente por una tupla de dos elementos: la Dirección IP de destino y el Número de puerto de destino

TCP es un protocolo **orientado a la conexión**. Su demultiplexación se considera **orientada a la conexión**.

• **Identificación del Socket (Conexión):** Un socket TCP, y por lo tanto una conexión TCP, se identifica de forma única por una **4-tupla**:

- **Dirección IP de origen**
- **Número de puerto de origen**
- **Dirección IP de destino**
- **Número de puerto de destino**

c. Orientado a la conexión.

Explicado arriba

d. Controles de congestión.

TCP: Incorpora mecanismos de control de congestión, utiliza un control de congestión terminal a terminal, infiriendo la congestión por la pérdida de segmentos

UDP: No incluye un mecanismo de control de congestión, una aplicación que emplea UDP puede enviar datos a la velocidad que le parezca, durante todo el tiempo que quiera, sin estar sujeta a las restricciones de velocidad impuestas por la red

e. Utilización de puertos.

TCP: Utiliza números de puerto de origen y destino, además de la IP de origen y destino, para identificar de forma única la conexión (la 4-tupla). La cabecera TCP tiene un tamaño típico de 20 bytes. Este tamaño es mayor debido a los campos necesarios para la fiabilidad, control de flujo y gestión de conexión.

UDP: Utiliza números de puerto para la función de multiplexación/demultiplexación (para dirigir el segmento al proceso apropiado)

5. La PDU de la capa de transporte es el segmento. Sin embargo, en algunos contextos suele utilizarse el término datagrama. Indique cuando.

◦ Para **TCP (Protocolo de Control de Transmisión)**, la PDU es el **Segmento**, que representa una porción del flujo de bytes.

◦ Para **UDP (Protocolo de Datagramas de Usuario)**, la PDU se denomina **datagrama** o **Segmento** (este último por coherencia con el nivel de Transporte)

6. Describa el saludo de tres vías de TCP. ¿UDP tiene esta característica?

Es el proceso de acuerdo en tres fases (three-way handshake) que TCP utiliza para establecer una conexión full-duplex entre dos *hosts* antes de que los procesos de aplicación puedan comenzar a enviar datos.

1. El cliente envía un segmento **SYN** (con el *bit SYN puesto a 1*) al servidor e incluye un número de secuencia inicial (cliente_nsi).
2. El servidor asigna buffers y variables, y responde con un segmento SYNACK. Este segmento tiene el bit SYN puesto a 1, su propio número de secuencia inicial (servidor_nsi) y el campo de reconocimiento (ACK) se fija en cliente_nsi + 1.
3. El cliente asigna recursos y envía el último segmento, que **confirma** el segmento SYNACK del servidor. Este segmento tiene el *bit SYN puesto a cero* y el campo de **reconocimiento (ACK)** se fija en **servidor_nsi + 1**.

No. UDP es un protocolo sin conexión. UDP no realiza ningún proceso para establecer una conexión antes de la transferencia de datos, lo que evita el retraso asociado al establecimiento de la conexión que requiere TCP.

7. Investigue qué es el ISN (Initial Sequence Number). Relaciónelo con el saludo de tres vías.

El ISN es el **Número de Secuencia Inicial** que cada host elige **aleatoriamente** al inicio de una conexión TCP.

Se utiliza para **numerar secuencialmente** los bytes de datos que fluirán del emisor hacia el receptor en una conexión TCP. El ISN marca el número de secuencia del primer byte que se enviará en ese flujo.

Se genera cuando ambos lados de una conexión TCP eligen aleatoriamente un número de secuencia inicial. Algunas implementaciones pueden utilizar un contador que se incrementa cada 4 milisegundos para generar el ISN

El saludo de tres vías es el proceso por el cual ambos *hosts* negocian y confirman sus respectivos ISN, que inicializan las variables de estado para la conexión (como los números de secuencia y reconocimiento), permitiendo la transferencia de datos fiable.

8. Investigue qué es el MSS. ¿Cuándo y cómo se negocia?

El MSS se define como la máxima cantidad de datos de la capa de aplicación que pueden ser incluidos en un segmento TCP. Es importante notar que el MSS se refiere únicamente a la **cantidad máxima de datos de la capa de aplicación dentro del segmento**, y no al tamaño total del segmento TCP, incluyendo sus cabeceras. Es un parámetro utilizado por TCP.

La negociación del MSS se lleva a cabo como parte de la **gestión de la conexión TCP**.

- **¿Cuándo?** La negociación ocurre **durante el establecimiento de la conexión TCP**, antes de que los procesos de aplicación comiencen a transferir datos.
- **¿Cómo?** El MSS se negocia utilizando el **campo Opciones** de la cabecera del segmento TCP. El campo Opciones es de longitud variable y se emplea para que el emisor y el receptor negocien este tamaño.

9. Utilice el comando ss (reemplazo de netstat) para obtener la siguiente información de su PC:

a. Para listar las comunicaciones TCP establecidas.

```
File Edit View Terminal Tabs Help
redes@debian:~$ ss -t state established
Recv-Q Send-Q Local Address:Port Peer Address:Port Process
```

Cabecera Significado resumido

Netid Protocolo usado por la conexión (TCP, UDP, UNIX, etc.).

State Estado actual de la conexión (por ej. ESTAB = establecida, LISTEN = esperando).

Recv-Q Bytes recibidos por el socket pero aún **no leídos** por la aplicación local.

Send-Q Bytes enviados por la aplicación pero aún **no confirmados** o no transmitidos.

Local Address:Port Dirección IP y puerto **local** (de tu máquina).

Peer Address:Port Dirección IP y puerto **remoto** (del otro extremo de la conexión).

Process Nombre del **proceso**, su **PID** y el **descriptor** de archivo asociado.

// por qué no aparecen Netid y State?

b. Para listar las comunicaciones UDP establecidas.

```
redes@debian:~$ ss -u state established
Recv-Q Send-Q Local Address:Port Peer Address:Port Process
0 0 192.168.100.87:bootpc 192.168.100.1:bootps
```

c. Obtener sólo los servicios TCP que están esperando comunicaciones

```
redes@debian:~$ ss -t state listening
Recv-Q Send-Q Local Address:Port Peer Address:Port Process
0 5 127.0.0.1:4038 0.0.0.0:*
0 128 0.0.0.0:ssh 0.0.0.0:*
0 128 127.0.0.1:ipp 0.0.0.0:*
0 4096 [::1]:50051 [::]:*
0 4096 [::ffff:127.0.0.1]:50051 *: *
0 128 [::]:ssh [::]:*
0 128 [::1]:ipp [::]:*
```

d. Obtener sólo los servicios UDP que están esperando comunicaciones.

```
redes@debian:~$ ss -u state listening
Recv-Q Send-Q Local Address:Port Peer Address:Port Process
```

e. Repetir los anteriores para visualizar el proceso del sistema asociado a la conexión.

f. Obtenga la misma información planteada en los items anteriores usando el comando netstat.

```
redes@debian:~$ netstat -tan state established
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp      0      0 127.0.0.1:4038          0.0.0.0:*               LISTEN
tcp      0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
tcp      0      0 127.0.0.1:631          0.0.0.0:*               LISTEN
tcp6     0      0 :::1:50051              :::*                   LISTEN
tcp6     0      0 127.0.0.1:50051        :::*                   LISTEN
tcp6     0      0 :::22                   :::*                   LISTEN
tcp6     0      0 :::1:631                :::*                   LISTEN
```

Aparecen los activos y los que están en espera, mostrando el estado.

```
redes@debian:~$ netstat -tuan state established
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
udp      0      0 127.0.0.1:4038          0.0.0.0:*
udp      0      0 192.168.100.87:68       192.168.100.1:67       ESTABLISHED
udp      0      0 0.0.0.0:5353            0.0.0.0:*
udp      0      0 0.0.0.0:33168           0.0.0.0:*
udp      0      0 0.0.0.0:631             0.0.0.0:*
udp6     0      0 :::5353                 :::*
udp6     0      0 fe80::a00:27ff:fe4e:546 :::*
udp6     0      0 :::38543                :::*
```

Lo mismo con el UDP.

10. ¿Qué sucede si llega un segmento TCP con el flag SYN activo a un host que no tiene ningún proceso esperando en el puerto destino de dicho segmento (es decir, el puerto destino no está en estado LISTEN)?

Quando un cliente quiere **iniciar una conexión TCP**, envía un **segmento con el flag SYN = 1**. El servidor solo responderá si **hay un proceso escuchando (LISTEN)** en ese puerto destino.

a. Utilice hping3 para enviar paquetes TCP al puerto destino 22 de la máquina virtual con el flag SYN activado.

```
--- localhost hping statistic ---
129 packets transmitted, 129 packets received, 0% packet loss
round-trip min/avg/max = 0.1/4.6/12.7 ms
redes@debian:~$
```

Eso sucedió al enviar tantos paquetes y no especificar cuanto era la cantidad. No se perdió ningún paquete.

b. Utilice hping3 para enviar paquetes TCP al puerto destino 40 de la máquina virtual con el flag SYN activado.

Lo mismo con el otro comando, sin paquetes perdidos.

c. ¿Qué diferencias nota en las respuestas obtenidas en los dos casos anteriores? ¿Puede explicar a qué se debe? (Ayuda: utilice el comando ss visto anteriormente).

// PREGUNTAR Y HACERLO DE NUEVO

11. ¿Qué sucede si llega un datagrama UDP a un host que no tiene ningún proceso esperando en el puerto destino de dicho datagrama (es decir, que dicho puerto no está en estado LISTEN)?

UDP es **no orientado a conexión** y **no confiable**, o sea:

- No hay handshake (no hay SYN/ACK).
- El host **no guarda estado de conexión**.
- Si llega un datagrama UDP a un puerto **cerrado**, el host **responde con un mensaje ICMP** (tipo 3, código 3):
→ “Port Unreachable” (puerto inaccesible).

a. Utilice hping3 para enviar datagramas UDP al puerto destino 5353 de la máquina virtual.

```
redes@debian:~$ sudo hping3 -2 -p 5353 -c 1 -V localhost
using lo, addr: 127.0.0.1, MTU: 65536
HPING localhost (lo 127.0.0.1): udp mode set, 28 headers + 0 data bytes

--- localhost hping statistic ---
1 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

No se recibe ningún paquete. Probé con más paquetes y tampoco.

b. Utilice hping3 para enviar datagramas UDP al puerto destino 40 de la máquina virtual.

Se reciben todos los datagramas correctamente.

c. ¿Qué diferencias nota en las respuestas obtenidas en los dos casos anteriores? ¿Puede explicar a qué se debe? (Ayuda: utilice el comando ss visto anteriormente).

// PREGUNTAR la diferencia

12. Investigue los distintos tipos de estado que puede tener una conexión TCP.

Estado	Significado
LISTEN	El servidor está esperando conexiones entrantes (socket en espera).
SYN-SENT	El cliente envió un SYN y espera el SYN+ACK del servidor (inicio del handshake).
SYN-RECV	El servidor recibió un SYN y respondió con SYN+ACK, esperando el ACK final.
ESTABLISHED	Conexión completamente establecida (los dos extremos pueden enviar y recibir datos).
FIN-WAIT-1	Un extremo envió FIN (cierre iniciado) y espera ACK o FIN del otro lado.
FIN-WAIT-2	El otro extremo confirmó el cierre inicial, pero no ha cerrado todavía.
CLOSE-WAIT	El host recibió un FIN del otro lado y debe responder con su propio FIN.
CLOSING	Ambos extremos están cerrando simultáneamente.

Estado	Significado
LAST-ACK	Espera el ACK final del otro extremo tras enviar su FIN.
TIME-WAIT	Espera un tiempo (2×MSL) para asegurar que no queden paquetes retrasados.
CLOSED	Conexión finalizada, sin socket asociado.

13. Dada la siguiente salida del comando ss, responda:

```

Netid State  Recv-Q Send-Q Local Address:Port      Peer Address:Port      Process
tcp LISTEN  0      128      *:22                      *:.*                     users:({ "sshd",pid=468,fd=29})
tcp LISTEN  0      128      *:80                      *:.*                     users:({ "apache2",pid=991,fd=95})
udp LISTEN  0      128      163.10.5.222:53          *:.*                     users:({ "named",pid=452,fd=10})
tcp ESTAB   0      0      163.10.5.222:59736       64.233.163.120:443      users:({ "x-www-browser",pid=1079,fd=51})
tcp CLOSE-WAI T 0      0      163.10.5.222:41654       200.115.89.30:443      users:({ "x-www-browser",pid=1079,fd=50})
tcp ESTAB   0      0      163.10.5.222:59737       64.233.163.120:443      users:({ "x-www-browser",pid=1079,fd=55})
tcp ESTAB   0      0      163.10.5.222:33583       200.115.89.15:443       users:({ "x-www-browser",pid=1079,fd=53})
tcp ESTAB   0      0      163.10.5.222:45293       64.233.190.99:443       users:({ "x-www-browser",pid=1079,fd=59})
tcp LISTEN  0      128      *:25                      *:.*                     users:({ "postfix",pid=627,fd=3})
tcp ESTAB   0      0      127.0.0.1:22             127.0.0.1:41220         users:({ "sshd",pid=1418,fd=3},
({ "sshd",pid=1416,fd=3})
tcp ESTAB   0      0      163.10.5.222:52952       64.233.190.94:443       users:({ "x-www-browser",pid=1079,fd=29})
tcp TIME-WAIT 0      0      163.10.5.222:36676       54.149.207.17:443       users:({ "x-www-browser",pid=1079,fd=3})
tcp ESTAB   0      0      163.10.5.222:52960       64.233.190.94:443       users:({ "x-www-browser",pid=1079,fd=67})
tcp ESTAB   0      0      163.10.5.222:50521       200.115.89.57:443       users:({ "x-www-browser",pid=1079,fd=69})
tcp SYN-SENT 0      0      163.10.5.222:52132       43.232.2.2:9500         users:({ "x-www-browser",pid=1079,fd=70})
tcp ESTAB   0      0      127.0.0.1:41220         127.0.0.1:22           users:({ "ssh",pid=1415,fd=3})
udp LISTEN  0      128      127.0.0.1:53            *:.*                     users:({ "named",pid=452,fd=9})

```

Después de los dos puntos (:) indica el puerto

a. ¿Cuántas conexiones hay establecidas?

9

b. ¿Cuántos puertos hay abiertos a la espera de posibles nuevas conexiones?

5, 3 TCP y 2 UDP.

c. El cliente y el servidor de las comunicaciones HTTPS (puerto 443), ¿residen en la misma máquina?

No, en el puerto 443 el Local Address es diferente al Peer Address.

d. El cliente y el servidor de la comunicación SSH (puerto 22), ¿residen en la misma máquina?

Si

e. Liste los nombres de todos los procesos asociados con cada comunicación. Indique para cada uno si se trata de un proceso cliente o uno servidor.

Preguntar cuáles serían los procesos.

f. ¿Cuáles conexiones tuvieron el cierre iniciado por el host local y cuáles por el remoto?

- **CLOSE-WAIT** (línea 5): indica que **el remoto envió el FIN y el host local no cerró aún completamente** → **cierre iniciado por el remoto**.
- **TIME-WAIT** (línea 12): indica que **el host local fue quien inició el cierre** (envió FIN) y ahora espera el timeout → **cierre iniciado por el local**.
- **SYN-SENT** (línea 15): intento de conexión en curso (no es cierre).
- Las conexiones **ESTAB** no están cerrándose ahora (no hay cierre iniciado aún).

g. ¿Cuántas conexiones están aún pendientes por establecerse?

1 con SYN – SENT, cuál es la diferencia con CLOSE-WAIT y TIME WAIT?

14. Dadas las salidas de los siguientes comandos ejecutados en el cliente y el servidor, responder:

```
servidor# ss -natu | grep 110
tcp    LISTEN 0 0          *:110      *:*
tcp    SYN-RCV  0 0          157.0.0.1:110 157.0.11.1:52843

cliente# ss -natu | grep 110
tcp    SYN-SENT 0 1          157.0.11.1:52843 157.0.0.1:110
```

a. ¿Qué segmentos llegaron y cuáles se están perdiendo en la red?

Llegó: el **SYN** enviado por el cliente llegó al servidor (eso explica el estado SYN-RCV en el servidor).

- **Se están perdiendo (probablemente):** el **SYN+ACK** enviado por el servidor hacia el cliente — el cliente no lo recibe y por eso sigue en SYN-SENT.
(Alternativa menos probable: el cliente recibe el SYN+ACK pero su **ACK final** se pierde; en ese caso el servidor seguiría en SYN-RCV y el cliente podría mostrar otro estado, pero la evidencia más directa apunta a pérdida del **SYN+ACK**.)

b. ¿A qué protocolo de capa de aplicación y de transporte se está intentando conectar el cliente?

Capa de Transporte: TCP, lo dice ahí. **Capa de Aplicación:** Puerto 110 (POP3)

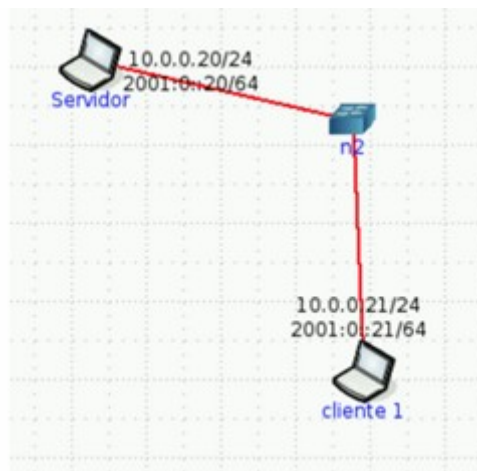
c. ¿Qué flags tendría seteado el segmento perdido?

El segmento que **probablemente se está perdiendo** es el que el servidor debería haber enviado como respuesta al SYN: lleva los flags **SYN + ACK** (es decir, SYN = 1 y ACK = 1).

15. Use CORE para armar una topología como la siguiente, sobre la cual deberá realizar:

a. En ambos equipos inspeccionar el estado de las conexiones y mantener abiertas ambas ventanas con el comando corriendo para poder visualizar los cambios a medida que se realiza el ejercicio.
Ayuda: `watch -n1 'ss -nat'`.

b. En Servidor, utilice la herramienta `ncat` para levantar un servicio que escuche en el puerto 8001/TCP. Utilice la opción `-k` para que el servicio sea persistente. Verifique el estado de las conexiones.



c. Desde CLIENTE1 conectarse a dicho servicio utilizando también la herramienta `ncat`. Inspeccione el estado de las conexiones.

d. Iniciar otra conexión desde CLIENTE1 de la misma manera que la anterior y verificar el estado de las conexiones. ¿De qué manera puede identificar cada conexión?

e. En base a lo observado en el ítem anterior, ¿es posible iniciar más de una conexión desde el cliente al servidor en el mismo puerto destino? ¿Por qué? ¿Cómo se garantiza que los datos de una conexión no se mezclarán con los de la otra?

f. Analice en el tráfico de red, los flags de los segmentos TCP que ocurren cuando:

i. Cierra la última conexión establecida desde CLIENTE1. Evalúe los estados de las conexiones en ambos equipos.

ii. Corta el servicio de `ncat` en el servidor (Ctrl+C). Evalúe los estados de las conexiones en ambos equipos.

iii. Cierra la conexión en el cliente. Evalúe nuevamente los estados de las conexiones.